INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
UNIDADE ACADÊMICA DE INFORMAÇÃO E COMUNICAÇÃO
COORDENAÇÃO DO CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

RELATÓRIO DE ESTÁGIO

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

João Pessoa - PB
10/2026

Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Unidade Acadêmica de Informação e Comunicação
Coordenação do Curso Superior de Tecnologia em Sistemas para Internet

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

Relatório de Estágio apresentado ao Coordenação do Curso Superior de Tecnologia em Sistemas para Internet da Instituto Federal de Educação, Ciência e Tecnologia da Paraíba, como requisito parcial para a obtenção do título de Tecnólogo.

Orientador:	Juliana Dantas Ribeiro Viana de Medeiros
Supervisor:	Juliana Dantas Ribeiro Viana de Medeiros
Coordenador do Curso:	Cândido José Ramos do Egypto
Empresa:	Agência de Inovação (NEO-IFPB)
Período:	25/02/2026 à 19/06/2026

João Pessoa - PB

APROVAÇÃO

Esta folha deverá ser substituída pela cópia digitalizada da folha de aprovação fornecida.

Juliana Dantas Ribeiro Viana de Medeiros
Supervisor da Empresa

Cândido José Ramos do Egypto
Coordenador do CST de Sistemas para Internet

Prof. Dr. Juliana Dantas Ribeiro Viana de Medeiros
Professor Orientador

Luis Kilmer da Silva Bernardo
Estagiário

Altere este texto inserindo a dedicatória do seu trabalho.

AGRADECIMENTOS

Edite e coloque aqui os agradecimentos às pessoas e/ou instituições que contribuíram para a realização do trabalho.

É obrigatório o agradecimento às instituições de fomento à pesquisa que financiaram total ou parcialmente o trabalho, inclusive no que diz respeito à concessão de bolsas.

Eu denomino meu campo de Gestão do Conhecimento, mas você não pode gerenciar conhecimento. Ninguém pode. O que pode fazer - o que a empresa pode fazer - é gerenciar o ambiente que otimize o conhecimento. (PRUSAK, Laurence, 1997).

RESUMO

Silva Bernardo, Luis Kilmer. Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas. 10/2026. 27 f. Relatório de Estágio – Coordenação do Curso Superior de Tecnologia em Sistemas para Internet, Instituto Federal de Educação, Ciência e Tecnologia da Paraíba. João Pessoa - PB, 10/2026.

Este relatório apresenta as atividades desenvolvidas no Laboratório Assert do Instituto Federal da Paraíba (IFPB), Campus João Pessoa, entre fevereiro e junho de 2026, por meio do PIAGI. O estágio teve como objetivo desenvolver um módulo para o sistema Banco de Especialistas, visando formalizar e rastrear solicitações de alterações cadastrais e de classificação dos usuários. Para isso, implementou-se o módulo de Solicitações, contemplando a modelagem das entidades Solicitação e Snapshot, as regras de negócio para criação, edição, aprovação e rejeição de solicitações e uma linha do tempo (timeline) para registrar os eventos e as decisões do processo. O desenvolvimento também abrangeu a implementação de operações de persistência e endpoints, a validação e serialização de dados, a integração com a arquitetura existente e a construção de funcionalidades de interface para solicitantes e avaliadores. A solução utilizou Python, Flask, PostgreSQL, SQLAlchemy e Marshmallow no back-end, Nuxt no front-end e Docker e Docker Compose para configuração do ambiente. Também foram desenvolvidos testes automatizados com Pytest para verificar as funcionalidades implementadas. Como resultado, o módulo integrou o fluxo de solicitações à plataforma, reduzindo a dependência de comunicações por e-mail e intervenções manuais no banco de dados, além de proporcionar maior formalização, rastreabilidade e histórico das alterações realizadas.

Palavras-chave: Desenvolvimento full-stack. Banco de Especialistas. Solicitações. Rastreabilidade. Testes automatizados.

LISTA DE FIGURAS

Figura 1 – Fluxo de solicitações manuais de ajuste cadastral e de categoria antes do desenvolvimento do módulo. 13

LISTA DE QUADROS

LISTA DE TABELAS

Tabela 1 – Volume de chamados por tipo de inscrição 13
Tabela 2 – Distribuição dos chamados por categoria de problema 13

LISTA DE ALGORITMOS

Algoritmo 1 – Padrão de implementação observado em controladores existentes. . . . 10
Algoritmo 2 – Estrutura do novo módulo, com separação de responsabilidades. 11

SUMÁRIO

LISTA DE FIGURAS viii

LISTA DE QUADROS ix

LISTA DE TABELAS x

LISTA DE ABREVIATURAS E SIGLAS xi

LISTA DE SÍMBOLOS xi

LISTA DE ALGORITMOS xi

1 – INTRODUÇÃO 1

1.1 Objetivos 1

1.1.1 Objetivo geral 1

1.1.2 Objetivos específicos 2

1.2 Escopo e limitações 2

1.3 Procedimentos metodológicos 2

1.4 Contexto institucional 3

1.5 Organização do relatório 3

2 – EMBASAMENTO TÉORICO 4

2.1 Arquitetura Web Cliente-Servidor 4

2.2 Flask 4

2.3 Persistência de dados com PostgreSQL e SQLAlchemy 4

2.3.1 PostgreSQL 5

2.3.2 SQLAlchemy 5

2.4 Validação de Dados com Marshmallow e Flasgger 5

2.5 Auditoria, Rastreabilidade e Versionamento de Dados 5

2.5.1 Auditoria e rastreabilidade 5

2.5.2 Fluxo de aprovação com máquina de estados 6

2.5.3 Snapshot e reversão 6

2.5.4 Timeline e registro de eventos 7

2.6 Verificação das Transições com Pytest 7

2.7 Ambiente Reprodutível de Testes com Docker 7

2.8 Controle de Versão e Gestão de Configuracao 7

2.9 Síntese do Capítulo 8

3 – ATIVIDADES REALIZADAS 9
3.1 Estudo da Arquitetura do Sistema Existente 9
3.1.1 Visão Geral do Sistema 9
3.1.2 Características do Código Existente e Decisão por Additive Change . 9
3.1.3 Análise das Regras de Negócio 12
3.1.4 Entendimento do Fluxo Antigo e Diagnóstico da Demanda 12
3.1.5 Levantamento de Requisitos das Funcionalidades 14
3.1.5.1 Requisitos Funcionais 15
3.1.5.2 Requisitos Não Funcionais 15
3.2 Desenvolvimento do Módulo de Solicitações 15
3.2.1 Endpoints 15
3.2.2 Integração com o banco de dados 15
3.2.3 Autorização e Controle de Acesso 15
3.2.4 Aprovação e Rejeição 15
3.2.5 Mudança de categoria 16
3.2.6 Atualização de pontuação 16
3.3 Implementação do Sistema de Snapshot e Auditoria 16
3.3.1 Entidade Snapshot 16
3.3.2 JSONB 16
3.3.3 Reversão 16
3.3.4 Timeline 16
3.3.5 Rastreabilidade 17
3.4 Testes e Validação 17
3.4.1 Pytest 17
3.4.2 Pré-configuração 17
3.4.3 Testes unitários 17
3.4.4 Testes de integração 17
3.4.5 Processo de Integração Contínua 17

4 – CONSIDERAÇÕES FINAIS 19

Referências 20

Apêndices 22

APÊNDICE A – Nome do apêndice 23

APÊNDICE B – Nome do outro apêndice 24

Anexos 25

ANEXO A–Nome do anexo 26

ANEXO B–Nome do outro anexo 27

1 INTRODUÇÃO

O presente relatório descreve as atividades desenvolvidas durante o estágio supervisionado realizado entre fevereiro e junho de 2026 no Laboratório Assert, vinculado ao Polo de Inovação do Instituto Federal da Paraíba (IFPB). As atividades concentraram-se na evolução do sistema Banco de Especialistas, plataforma destinada ao gerenciamento de especialistas e ao apoio à composição de equipes para projetos de Pesquisa, Desenvolvimento e Inovação (PD&I).

O Banco de Especialistas permite o cadastro, a classificação e a avaliação de candidatos com base em critérios como formação acadêmica, experiência profissional e participação em projetos. Nesse contexto, a plataforma reúne informações utilizadas na gestão dos especialistas e nos processos de seleção e composição de equipes vinculadas às iniciativas do Polo de Inovação.

Durante o estágio, identificou-se uma demanda relacionada à alteração de categoria dos especialistas. O procedimento então utilizado exigia que o interessado solicitasse a mudança por e-mail e que um administrador realizasse manualmente uma consulta SQL diretamente no banco de dados. A ausência de um fluxo formal de solicitações dificultava o acompanhamento das alterações e a recuperação do estado anterior dos registros, além de aumentar o risco de inconsistências decorrentes de intervenções diretas na base de dados.

Diante desse cenário, o trabalho concentrou-se no desenvolvimento de um módulo de solicitações para formalizar e gerenciar operações de alteração cadastral e de classificação dos especialistas. A solução contempla mecanismos para registrar solicitações, acompanhar seu andamento, permitir sua análise e decisão e manter informações necessárias à rastreabilidade das operações. Dessa forma, busca-se reduzir a dependência de intervenções manuais no banco de dados e estabelecer um processo mais controlado e verificável.

Do ponto de vista técnico, o desenvolvimento envolveu conceitos de engenharia de software, aplicações web, persistência de dados relacionais, testes automatizados e conteinerização. Destacam-se a integração entre componentes de front-end e back-end, o uso de mecanismos de registro e recuperação do estado dos usuários e a manutenção de um histórico das operações. Esses fundamentos são apresentados no capítulo dedicado ao embasamento teórico.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver um módulo de solicitações de alteração cadastral e de classificação no sistema Banco de Especialistas, contemplando os fluxos de criação, consulta, aprovação, rejeição e reversão, com mecanismos de rastreabilidade e registro das operações.

1.1.2 Objetivos específicos

- Levantar e analisar os requisitos e as regras de negócio relacionados às solicitações de alteração cadastral e de classificação dos especialistas.
- Modelar os componentes de persistência necessários ao registro das solicitações e à recuperação do estado dos usuários.
- Implementar as funcionalidades de gerenciamento das solicitações no back-end, incluindo as regras de negócio, a integração com o PostgreSQL e o acesso aos dados por meio do SQLAlchemy.
- Desenvolver as interfaces web necessárias à criação, ao acompanhamento e à análise das solicitações.
- Implementar mecanismos de rastreabilidade das operações, incluindo o registro de eventos e a apresentação de seu histórico.
- Desenvolver e executar testes unitários e de integração para verificar as regras de negócio e o funcionamento integrado dos componentes da aplicação.

1.2 Escopo e limitações

O trabalho abrange o desenvolvimento do módulo de solicitações e as funcionalidades necessárias ao gerenciamento das operações de alteração cadastral e de classificação dos especialistas no sistema Banco de Especialistas. Estão incluídos os mecanismos de registro, acompanhamento, decisão e reversão das solicitações, bem como os recursos de persistência e rastreabilidade associados a esse fluxo.

Não fazem parte do escopo a redefinição dos critérios de pontuação dos especialistas, a modificação do processo seletivo, a substituição do sistema de avaliação existente e a implementação de notificações por e-mail. Também não é contemplado o tratamento de múltiplas solicitações simultâneas para um mesmo especialista. Essas limitações delimitam as responsabilidades da solução desenvolvida e podem orientar evoluções futuras do sistema.

1.3 Procedimentos metodológicos

O trabalho caracteriza-se como um relato técnico de natureza aplicada e caráter descritivo, desenvolvido no contexto de um estágio supervisionado em desenvolvimento de software. Sua execução compreendeu o estudo da arquitetura e das regras de negócio do sistema, o levantamento e a análise dos requisitos, a modelagem dos componentes necessários, a implementação das funcionalidades e a realização de testes.

A verificação técnica envolveu testes unitários e de integração, destinados a avaliar o comportamento das funcionalidades implementadas e a interação entre os componentes da aplicação. Também foi verificado o funcionamento integrado do front-end e do back-end em ambiente de desenvolvimento conteinerizado com Docker. As atividades foram conduzidas no

contexto da arquitetura e das tecnologias já adotadas pelo projeto, considerando suas restrições e regras de negócio.

As atividades do módulo foram organizadas no OpenProject (OpenProject, 2026), com issues, responsáveis e prazos, o que apoiou a priorização das tarefas e o acompanhamento do progresso ao longo do estágio.

1.4 Contexto institucional

O estágio foi realizado no âmbito do Polo de Inovação do Instituto Federal da Paraíba, estrutura associada à Agência de Inovação da instituição (NEO-IFPB). O Polo atua no desenvolvimento de iniciativas de Pesquisa, Desenvolvimento e Inovação (PD&I), aproximando a instituição de demandas tecnológicas do setor produtivo e da sociedade.

As atividades técnicas foram desenvolvidas no Laboratório Assert, ambiente vinculado ao Polo de Inovação no qual são realizadas atividades de desenvolvimento e manutenção de sistemas computacionais para projetos institucionais. Nesse contexto, o estágio proporcionou a participação nas etapas de análise, implementação, integração e verificação de funcionalidades de uma aplicação web existente.

1.5 Organização do relatório

Além desta introdução, o relatório está organizado em três capítulos. O capítulo 2 apresenta os conceitos e as tecnologias que fundamentam o módulo. O capítulo 3 descreve as atividades do estágio, da análise dos requisitos à implementação e aos testes. O capítulo 4 reúne as considerações finais, com a síntese dos resultados, as limitações e as possibilidades de evolução do trabalho.

2 EMBASAMENTO TÉORICO

Este capítulo apresenta, de forma clara, os conceitos e as tecnologias que fundamentam as atividades descritas neste relatório. Como o estágio teve por foco o desenvolvimento de um módulo de solicitações com a rastreabilidade, auditoria e reversão para o sistema Banco de Especialistas, é dada ênfase ao *back-end* (Flask, SQLAlchemy e PostgreSQL) e aos conceitos de auditoria e versionamento de dados. As demais tecnologias (Vue.js, Nuxt, Pytest, Git e Docker) são abordadas como elementos de apoio ao desenvolvimento, à validação e à execução do módulo.

2.1 Arquitetura Web Cliente-Servidor

A arquitetura cliente-servidor organiza um sistema distribuído em provedores de serviços e requisitantes (TANENBAUM; van Steen, 2007). Nesse contexto, a comunicação orientada pelo estilo REST e pelo protocolo HTTP define interações sem estado, nas quais cada requisição é independente e deve carregar as informações necessárias ao seu processamento (FIELDING; NOTTINGHAM; RESCHKE, 2022).

Essa base teórica fundamenta a separação de responsabilidades do módulo: isolar a máquina de estados e as regras de persistência no servidor garante que a integridade dos dados e as transições de status da solicitação não dependam do cliente. Dessa forma, a aplicação no frontend atua exclusivamente na apresentação e na coleta de informações, consumindo a API REST para criar, acompanhar e analisar as requisições.

2.2 Flask

O Flask foi utilizado na API do módulo por permitir incorporar suas rotas à aplicação existente sem impor uma reorganização do sistema. Os *blueprints* agruparam as operações de criação, análise, acompanhamento e reversão das solicitações em um componente isolado, preservando as demais funcionalidades e facilitando a manutenção do módulo (PALLETS, 2024).

2.3 Persistência de dados com PostgreSQL e SQLAlchemy

O módulo utiliza PostgreSQL para armazenar solicitações, alterações cadastrais e registros de auditoria. A persistência dessas informações precisa manter o cadastro e o histórico coerentes, sobretudo quando uma decisão altera mais de uma entidade.

2.3.1 PostgreSQL

O PostgreSQL foi adequado por oferecer transações para as operações compostas do fluxo. Na aprovação, a mudança de categoria, a atualização da pontuação, a gravação do *snapshot* do estado anterior e a atualização do status da solicitação devem ser confirmadas em conjunto; se uma etapa falhar, nenhuma alteração deve permanecer aplicada (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024).

2.3.2 SQLAlchemy

O SQLAlchemy foi adotado como ORM para coordenar as alterações de entidades relacionadas em uma única unidade de trabalho (BAYER, 2012). A sessão acumula as mudanças e, no *commit*, efetiva-as na transação; em caso de falha, o *rollback* descarta o conjunto. Assim, a aprovação não deixa a categoria atualizada sem o *snapshot* correspondente ou sem o novo status da solicitação.

2.4 Validação de Dados com Marshmallow e Flasgger

Na fronteira da API, os *schemas* do Marshmallow validam os campos e formatos recebidos antes que os dados cheguem às regras de negócio (MARSHMALLOW DEVELOPERS, 2024). Ao declarar os formatos esperados para as solicitações e suas respostas, também estabelecem o contrato consumido pelas interfaces de criação, acompanhamento e análise do módulo.

Complementarmente, o Flasgger foi utilizado na documentação e na especificação dos endpoints da API, permitindo descrever os parâmetros, os formatos esperados e as respostas das operações segundo as especificações OpenAPI/Swagger. Além da documentação interativa, a biblioteca oferece mecanismos de validação das requisições recebidas com base nos schemas definidos, desde que habilitados na configuração da aplicação. Dessa forma, sua utilização pode contribuir tanto para a explicitação do contrato da API quanto para a verificação da conformidade dos dados de entrada com as estruturas especificadas (FLASGGER DEVELOPERS, 2024).

2.5 Auditoria, Rastreabilidade e Versionamento de Dados

Esta seção reúne os conceitos que sustentam a principal contribuição do módulo: a capacidade de registrar, consultar e desfazer alterações cadastrais com garantias de rastreabilidade.

2.5.1 Auditoria e rastreabilidade

Auditoria, no contexto de sistemas de informação, é o registro sistemático de quem realizou determinada operação, quando, sobre qual dado e com qual resultado. A rastreabilidade

é a propriedade que permite reconstituir o histórico de uma informação a partir desses registros. Fowler (FOWLER, 2004) descreve o padrão *Audit Log* como um registro de alterações feitas nos dados, com o objetivo de permitir verificar o que ocorreu e, quando necessário, compreender a origem de um estado atual. Em sistemas que envolvem decisões de classificação e seleção de pessoas, como o Banco de Especialistas, esse registro contribui para a transparência e para a responsabilização dos agentes envolvidos.

O fluxo anterior ao módulo, baseado em e-mail e execução manual de SQL, ilustra a ausência dessas propriedades: não havia registro formal do solicitante, da justificativa, do responsável pela decisão ou do estado anterior do cadastro. Ao formalizar a solicitação como uma entidade persistente, com status, autoria e datas, o sistema passa a produzir esse registro como efeito natural do próprio fluxo de trabalho.

2.5.2 Fluxo de aprovação com máquina de estados

Uma solicitação percorre um ciclo de vida bem definido, que pode ser modelado como uma máquina de estados finitos: um conjunto de estados (por exemplo, pendente, aprovada e rejeitada) e de transições permitidas entre eles (SOMMERVILLE, 2011). Explicitar as transições válidas evita estados incoerentes: uma solicitação rejeitada não pode ser aprovada sem nova análise, e somente uma solicitação aprovada pode ser revertida. A validação dessas regras no servidor garante que o fluxo seja respeitado independentemente da interface utilizada.

2.5.3 Snapshot e reversão

Para possibilitar a reversão de uma alteração, é necessário preservar o estado anterior dos dados. O conceito de *snapshot* (instantâneo) consiste em capturar e armazenar o estado de um objeto em determinado momento. Ele se aproxima do padrão *Memento*, descrito por Gamma et al. (GAMMA et al., 1994), cujo propósito é capturar o estado interno de um objeto, sem violar seu encapsulamento, para que ele possa ser restaurado posteriormente.

No módulo, a entidade *Snapshot* registra o estado relevante do especialista (como categoria e pontuação) antes da aplicação de uma alteração aprovada. A reversão consiste em restaurar esses valores, mantendo a consistência entre categoria e pontuação. Cabe distinguir essa abordagem da simples exclusão do registro de histórico: a reversão é, ela mesma, uma operação registrada, de modo que o histórico permanece completo e a operação inversa pode ser auditada. Esse comportamento é análogo à ideia de transações compensatórias, em que o efeito de uma operação já efetivada é desfeito por uma nova operação, e não pela remoção do passado.

Existem abordagens mais gerais para o versionamento de dados, como o *Event Sourcing*, em que o estado é derivado de uma sequência imutável de eventos (FOWLER, 2005; KLEPPMANN, 2017). O módulo adota uma solução mais simples e proporcional ao escopo, combinando *snapshots* do estado anterior com um registro de eventos, e o conceito de *Event Sourcing* é aqui mencionado como referência conceitual para a ideia de histórico imutável.

2.5.4 Timeline e registro de eventos

A *timeline* é a apresentação ordenada cronologicamente dos eventos associados a uma entidade. Cada evento (criação, aprovação, rejeição, reversão) é registrado com seu tipo, autor, data e informações complementares, e a interface os exibe como uma linha do tempo. Trata-se de uma forma de tornar a auditoria acessível ao usuário final, e não apenas ao administrador do banco de dados.

2.6 Verificação das Transições com Pytest

O Pytest foi empregado para verificar as regras de negócio do módulo, em especial as transições permitidas da máquina de estados: criação, aprovação, rejeição e reversão (HOLGER KREKEL AND PYTEST DEVELOPERS, 2024). Os testes também verificaram os efeitos dessas transições sobre a categoria, a pontuação e o histórico da solicitação.

2.7 Ambiente Reprodutível de Testes com Docker

O Docker foi utilizado para reproduzir o ambiente necessário aos testes de integração, incluindo a API e o PostgreSQL, de modo que a validação dos fluxos ocorresse sob uma configuração consistente entre execuções (DOCKER, 2024).

2.8 Controle de Versão e Gestão de Configuracao

A gestão de configuração de software compreende a identificação, o controle e o registro das alterações realizadas nos artefatos de um projeto ao longo do tempo, de modo que seja possível reconstituir versões anteriores e identificar a origem de cada mudança (SOMMERVILLE, 2011). O controle de versão é o mecanismo que concretiza esse registro sobre o código-fonte: cada alteração é capturada em um commit, com autor, data e descrição, e o histórico resultante permite recuperar estados anteriores do sistema.

O módulo utilizou Git (Git, 2026) para esse fim, com ramificações (branches) por funcionalidade, o que isolou o desenvolvimento do módulo das demais partes do Banco de Especialistas até que as alterações fossem verificadas. Os repositórios foram hospedados no GitLab (GitLab, 2026), adotado como padrão pelo Laboratório Assert para os projetos do Polo de Inovação, e nele foi configurada uma pipeline de integração contínua que executa a suíte de testes (Pytest) a cada envio, impedindo que alterações que quebrem as regras de negócio sejam incorporadas à base principal.

Cabe observar que o controle de versão atua sobre o código-fonte, e não sobre os dados da aplicação. A rastreabilidade das alterações cadastrais dos especialistas, tratada na seção 2.5, é um problema distinto, resolvido no domínio da aplicação por meio de solicitações, snapshots e registro de eventos.

2.9 Síntese do Capítulo

As tecnologias e os conceitos apresentados compõem a base sobre a qual o módulo de solicitações foi desenvolvido. A arquitetura cliente-servidor e o estilo REST definem a forma de comunicação entre as interfaces e a API; o Flask estrutura a aplicação, o SQLAlchemy e o PostgreSQL garantem a persistência transacional, e o Marshmallow assegura a validação e o contrato dos dados. Os conceitos de auditoria, máquina de estados, *snapshot* e registro de eventos sustentam os requisitos de rastreabilidade e reversão que motivaram o estágio. Por fim, o Pytest garante a verificação das regras de negócio, o Git organiza a colaboração e o Docker fornece o ambiente reprodutível de execução. O próximo capítulo descreve como esses elementos foram aplicados nas atividades realizadas.

3 ATIVIDADES REALIZADAS

Este capítulo apresenta as principais atividades realizadas no decorrer do estudo e do desenvolvimento da funcionalidade de alteração de categoria de especialistas. A partir da análise do sistema existente e do levantamento das regras de negócio, foi possível compreender o contexto institucional, os fluxos de operação e os requisitos necessários para a implementação de uma solução integrada, auditável e compatível com a arquitetura atual.

3.1 Estudo da Arquitetura do Sistema Existente

3.1.1 Visão Geral do Sistema

O primeiro passo para o desenvolvimento da funcionalidade consistiu na compreensão do sistema Banco de Especialistas, de sua finalidade institucional e de sua organização técnica. A plataforma permite o cadastramento e a classificação de profissionais interessados em participar de projetos de pesquisa, desenvolvimento e inovação vinculados ao IFPB.

Nesse contexto, foram estudadas as funcionalidades relacionadas ao cadastro dos especialistas, à organização por categorias e aos critérios de classificação utilizados pelo sistema. Também foram examinados os componentes da aplicação e suas interações, com o objetivo de identificar os pontos de integração necessários ao desenvolvimento do novo módulo.

3.1.2 Características do Código Existente e Decisão por Additive Change

A adoção do princípio de Additive Change foi uma decisão arquitetural relevante para garantir a compatibilidade com a base já existente. Esse princípio orienta a evolução do sistema por meio da adição de novas rotas, campos, serviços e módulos, sem a necessidade de reescrever estruturas centrais ou alterar de forma profunda o comportamento de componentes já consolidados.

Essa estratégia se mostrou adequada ao contexto do projeto, pois a aplicação legada foi estruturada em um padrão MVC com controladores responsáveis por múltiplas tarefas, incluindo validação, persistência, tratamento de exceções, registro de eventos e comunicação com camadas externas. Esse desenho favoreceu a rapidez inicial de implementação, mas também resultou em baixa coesão, acoplamento funcional e maior dificuldade de manutenção e evolução.

A partir dessa análise, a nova funcionalidade foi concebida como uma extensão incremental do sistema, preservando os comportamentos já consolidados e adicionando um fluxo específico para a solicitação, aprovação, auditoria e registro histórico das mudanças de categoria. Essa abordagem reduziu o risco de regressão e permitiu a inserção de novos mecanismos sem comprometimento da infraestrutura preexistente.

A seguir, é apresentado o padrão observado em controladores existentes, evidenciando as características que motivaram a escolha por uma evolução incremental.

Algoritmo 1: Padrão de implementação observado em controladores existentes.

```
@bp.route('', methods=['POST'])
@swag.validate('UserRegistration')
def register():
    if not validate_cpf(request.json['cpf']):
        raise ValidationError(details={'cpf': 'invalid'})

    user = User(**request.json)
    user.hash_password()

    try:
        db.session.add(user)
        db.session.commit()
    except IntegrityError as e:
        msg = e.args[0]
        if 'users_tipo_especialista_fkey' in msg:
            raise InvalidIdFieldError('specialist_type_id')
        elif 'users_cpf_key' in msg:
            raise UniqueIdentifierAlreadyExistsError('cpf')
        elif 'users_email_key' in msg:
            raise UniqueIdentifierAlreadyExistsError('email')
        else:
            raise e

    create_timeline_log(
        activity_type_id=ActivityTypeEnum.CADASTRADO,
        user_id=user.id)

    msg = 'Informe seus dados pessoais...'
    create_notification(title='Boas vindas!', message=msg, user=user)

    return UserSchema().dump(user), HTTPStatus.CREATED
```

O padrão exemplificado no Algoritmo 1 demonstra a forma como os controladores legados eram organizados. Em uma mesma função, eram concentradas validação de entrada, construção do objeto de domínio, hash de senha, persistência no banco de dados, tratamento de erros de integridade, registro de timeline, envio de notificação e serialização da resposta. A validação dos dados de entrada era realizada pelo Flasgger, por meio do decorador `@swag.validate`, que verificava os dados conforme o schema associado à operação. Assim, a documentação Swagger também era utilizada como base para a validação das requisições.

Esse desenho gera alguns acoplamentos relevantes. O primeiro deles está no tratamento de erro, em que a lógica depende de strings de constraint do banco, como `users_tipo_especialista_fkey` e `users_cpf_key`. Esse tipo de condição cria uma forte dependência com o esquema físico do banco de dados, de forma que qualquer renomeação ou

alteração da constraint pode quebrar o fluxo de tratamento de exceções. O segundo acoplamento está na execução de operações colaterais após o commit, como a criação de registros de timeline e notificações. Nesse cenário, a criação do usuário e os eventos associados não são tratados como uma unidade transacional, de modo que falhas nas etapas posteriores podem deixar um registro principal persistido sem os dados complementares esperados.

A comparação com o novo módulo evidencia a importância da abordagem incremental. O desenvolvimento do módulo de solicitações foi organizado para separar responsabilidades e encapsular regras de negócio em serviços e camadas mais específicas, preservando o restante da aplicação e reduzindo o risco de efeitos colaterais.

Algoritmo 2: Estrutura do novo módulo, com separação de responsabilidades.

```
@bp_solicitacoes.route('', methods=['POST'])
@schema_required(SolicitacaoCreateSchema)
def criar_solicitacao(payload):
    solicitacao = solicitacao_service.criar(
        solicitante_id=current_user.id,
        tipo=payload['tipo'],
        dados=payload['dados'],
    )
    return SolicitacaoSchema().dump(solicitacao), HTTPStatus.CREATED


def criar(solicitante_id, tipo, dados):
    campos = SCHEMA_POR_TIPO[tipo].load(dados)
    solicitacao = Solicitacao(...)

    with db.session.begin_nested():
        db.session.add(solicitacao)
        registrar_evento(solicitacao, Evento.CRIADA)

    return solicitacao
```

O Algoritmo 2 ilustra o padrão adotado no novo módulo. A validação ocorre na fronteira, por meio do decorador `@schema_required(SolicitacaoCreateSchema)`, de modo que o serviço recebe um payload já validado. O controlador limita-se a delegar a execução ao serviço e serializar a resposta, sem concentrar hash, tratamento de erro do banco ou envio de notificações. A seleção do schema por tipo, por meio de `SCHEMA_POR_TIPO[tipo].load(dados)`, representa o mecanismo das restrições dinâmicas descritas na análise de regras de negócio e reforça a separação entre validação e persistência. Por fim, a persistência e o registro do evento são agrupados em uma mesma unidade de trabalho, eliminando o intervalo entre o commit e os efeitos colaterais observado no Algoritmo 1.

A escolha por Additive Change permitiu incorporar o novo fluxo sem reescrever os controladores existentes, preservando o comportamento consolidado da aplicação.

3.1.3 Análise das Regras de Negócio

A análise das regras de negócio concentrou-se nos critérios de classificação dos especialistas, nas alterações cadastrais e nas permissões associadas às operações de solicitação, aprovação e rejeição. O levantamento dessas regras permitiu estabelecer os requisitos comportamentais do novo módulo, considerando as restrições existentes e os possíveis impactos das operações sobre os registros do sistema.

Um dos principais desafios identificados decorre da organização do modelo de dados da aplicação: a categoria do especialista é definida no modelo *User*, e não diretamente no modelo *Specialist*. Embora os especialistas compartilhem uma estrutura de usuário comum, suas categorias determinam especializações distintas, com atributos específicos para cada tipo. Essa organização exige que as operações de transição considerem as relações entre os modelos envolvidos, de modo a preservar a integridade dos dados durante a alteração cadastral.

Outro aspecto relevante consiste em assegurar a rastreabilidade das alterações e a possibilidade de preservar o estado anterior dos registros durante o fluxo de solicitação. Para isso, é necessário distinguir o registro de uma solicitação da efetivação da mudança, condicionando esta última ao resultado do processo de análise. Adicionalmente, as transições devem considerar os critérios de classificação e pontuação aplicáveis a cada categoria, a fim de evitar inconsistências decorrentes da alteração dos dados cadastrais.

3.1.4 Entendimento do Fluxo Antigo e Diagnóstico da Demanda

Antes do desenvolvimento do módulo, as solicitações de alteração de categoria e de atualização cadastral eram realizadas exclusivamente por e-mail. Após o recebimento do pedido, a alteração dependia da intervenção de um membro da equipe de suporte com acesso privilegiado, que executava manualmente os comandos SQL correspondentes diretamente na base de dados, conforme ilustrado na Figura 1.

Esse procedimento estabelecia uma dependência entre a solicitação do usuário e a disponibilidade de um responsável com acesso ao banco. Além disso, o acompanhamento dos pedidos, a validação das justificativas e o histórico das alterações ficavam dispersos na caixa de entrada do suporte, fora de um fluxo centralizado e auditável.

Para dimensionar a recorrência desse processo, realizou-se o levantamento dos chamados recebidos pela caixa de suporte no período de 30 de maio de 2025 a 4 de agosto de 2026. A catalogação foi feita manualmente pela equipe, a partir do assunto e do conteúdo das mensagens, resultando em 141 chamados relacionados à gestão de cadastro e de acesso dos especialistas. A Tabela 1 apresenta a distribuição conforme o tipo de vínculo do usuário.

A concentração de 56,0% em colaboradores externos é relevante: trata-se do público com menor familiaridade com os procedimentos internos do IFPB e, portanto, mais dependente de um canal formal e autoexplicativo para solicitar ajustes.

A Tabela 2 detalha os motivos apresentados nesses atendimentos.

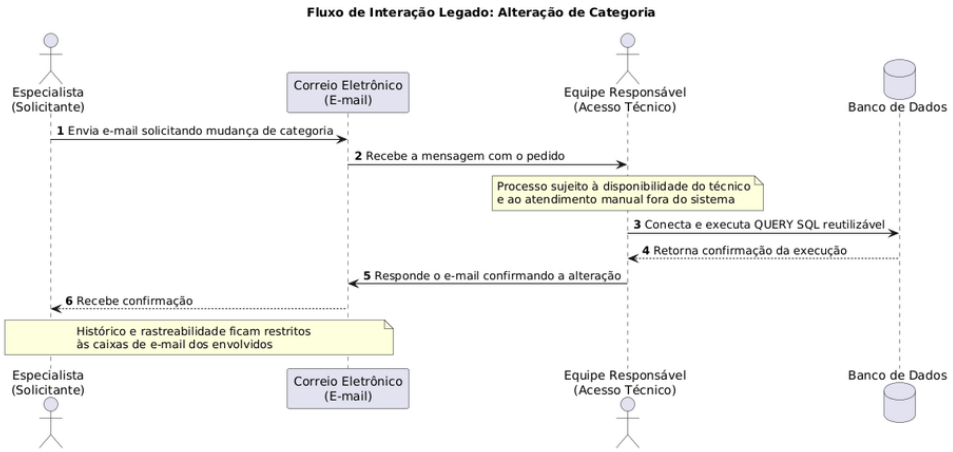


Figura 1 – Fluxo de solicitações manuais de ajuste cadastral e de categoria antes do desenvolvimento do módulo.

Fonte: Elaboração própria (2026).

Tabela 1 – Volume de solicitações de suporte por tipo de inscrição (maio/2025 a agosto/2026).

Tipo de Inscrição	**Quantidade de Chamados**	**Porcentagem (%)**
Colaborador(a) externo	79	56,0%
Estudante	47	33,3%
Servidor(a)	15	10,6%
Total	**141**	**100,0%**

Fonte: Elaboração própria, com base em dados do suporte (2026).

Tabela 2 – Distribuição das solicitações de suporte por categoria de problema.

Categoria do Problema	**Quantidade**	**Porcentagem (%)**
Outro (especificar)	59	41,8%
Completar cadastro (dados pessoais)	34	24,1%
Cadastrar pontuação	34	24,1%
Cadastrar no sistema	7	5,0%
Fazer login no sistema	7	5,0%
Total	**141**	**100,0%**

Fonte: Elaboração própria, com base em dados do suporte (2026).

A categoria "Outro" agrega solicitações heterogêneas que não foram classificadas em uma categoria específica no levantamento original, incluindo pedidos de alteração de categoria do especialista. Ainda que não tenha sido possível isolar cada uma dessas demandas na catalogação, observa-se que quase metade dos chamados (48,2%) corresponde a ajuste cadastral ou de pontuação, o que evidencia que o volume de trabalho manual não decorria apenas de casos isolados, mas de operações recorrentes de manutenção de dados.

A execução manual de comandos SQL em ambiente de produção, para atender a esse tipo de demanda, apresentava riscos que não se restringiam ao volume de chamados:

- **Risco de inconsistência:** execução manual sujeita a erro humano, sem validação automática das regras de negócio;
- **Ausência de rastreabilidade:** falta de registro sobre quem solicitou, quem aprovou e qual era o estado anterior do dado alterado;
- **Ineficiência operacional:** alocação de esforço técnico em tarefas repetitivas de suporte administrativo;
- **Dependência da equipe de TI:** indisponibilidade do responsável com acesso ao banco interrompia o atendimento.

O diagnóstico evidenciou a necessidade de substituir o fluxo manual por um módulo integrado ao sistema, capaz de formalizar as solicitações, submetê-las a uma máquina de estados para aprovação e rejeição (descrita na Seção 2.5.2) e manter histórico auditável das alterações. O módulo foi concebido para absorver três tipos de demanda identificados no levantamento: alteração de categoria, cadastro de pontuação e complemento cadastral. A alteração de categoria, por envolver dados do usuário e exigir possibilidade de reversão, recebeu tratamento específico, com registro de snapshot e linha do tempo, conforme detalhado na Seção 3.3.

3.1.5 Levantamento de Requisitos das Funcionalidades

É relevante destacar que o sistema deve preservar o comportamento anterior, sem alterar funcionalidades já existentes, mas incorporando novos módulos e processos que ampliem a capacidade operacional, a rastreabilidade e a segurança das alterações. A solução foi pensada para garantir que futuras mudanças de categoria e atualização de pontuação ocorram de forma controlada, com registro claro de cada ação e possibilidade de reversão em caso de inconsistências.

Além disso, o módulo deve respeitar as regras de negócio do sistema atual, incluindo critérios de aprovação, validação de dados e manutenção da integridade das informações de especialistas e usuários. Nesse contexto, a funcionalidade deve ser auditável, permitindo identificar quem solicitou a mudança, quem aprovou ou rejeitou a ação e quais dados foram alterados em cada etapa do processo.

3.1.5.1 Requisitos Funcionais

Os requisitos funcionais incluem: criação de solicitações de mudança de categoria, aprovação ou rejeição das solicitações, atualização de pontuação conforme critérios definidos, manutenção do histórico das mudanças e registro de ações relevantes para auditoria. O sistema também deve assegurar que somente usuários autorizados realizem operações de alteração, preservando a integridade dos dados e o fluxo operacional já estabelecido.

3.1.5.2 Requisitos Não Funcionais

Os requisitos não funcionais envolvem a necessidade de compatibilidade com a arquitetura existente, rastreabilidade das decisões, proteção contra alterações indevidas, clareza no acompanhamento do processo e facilidade de manutenção do código. A implementação também deve considerar a escalabilidade do módulo e a capacidade de auditoria por meio de snapshots e timeline de alterações.

3.2 Desenvolvimento do Módulo de Solicitações

3.2.1 Endpoints

Os endpoints foram desenvolvidos para oferecer uma interface clara e organizada para a criação, consulta e atualização das solicitações de mudança de categoria. A estrutura foi pensada para manter compatibilidade com a arquitetura existente, adicionando rotas específicas sem impactar o comportamento dos módulos já em funcionamento.

3.2.2 Integração com o banco de dados

A integração com o banco de dados foi planejada para armazenar as solicitações, os estados do processo e as informações necessárias à rastreabilidade. A persistência foi organizada para manter a relação entre a solicitação, o especialista e o histórico de decisões, garantindo consistência e confiabilidade no armazenamento das informações.

3.2.3 Autorização e Controle de Acesso

A autorização foi implementada considerando os diferentes papéis envolvidos no fluxo, de modo que apenas usuários com permissão adequada possam aprovar, rejeitar ou alterar as informações relacionadas à categoria do especialista. Esse controle é essencial para preservar as regras de negócio e evitar ações indevidas no sistema.

3.2.4 Aprovação e Rejeição

A funcionalidade de aprovação e rejeição foi estruturada como etapas distintas, cada uma com consequências específicas sobre o estado da solicitação. A aprovação formaliza a

decisão e desencadeia a mudança de categoria, enquanto a rejeição mantém o registro do pedido sem alteração do estado do especialista.

3.2.5 Mudança de categoria

A mudança de categoria representa o ponto central do módulo, sendo executada apenas quando a solicitação é aprovada e todas as regras pertinentes são satisfeitas. Esse processo foi implementado de forma a preservar a integridade dos dados e a lógica do sistema, evitando alterações abruptas ou inconsistentes em cadastros existentes.

3.2.6 Atualização de pontuação

A atualização de pontuação foi tratada como parte do processo de alteração de categoria, considerando os critérios de classificação previamente definidos. Essa etapa garante que a nova categoria reflita corretamente o histórico e os dados atualizados do especialista, mantendo a coerência da classificação institucional.

3.3 Implementação do Sistema de Snapshot e Auditoria

3.3.1 Entidade Snapshot

A entidade Snapshot foi concebida para registrar um estado completo do registro em um determinado momento, permitindo preservar informações relevantes antes de qualquer alteração. Essa abordagem torna possível reconstituir o contexto do dado em caso de revisão, validação ou correção posterior.

3.3.2 JSONB

O uso de JSONB foi adotado para armazenar os dados do snapshot em um formato estruturado e flexível, permitindo a persistência de informações detalhadas sem perder a capacidade de consulta e análise. Essa escolha favorece a rastreabilidade e facilita a recuperação do contexto em operações de auditoria.

3.3.3 Reversão

A reversão é um dos principais benefícios do sistema de snapshot, pois permite retornar o sistema a um estado anterior quando há inconsistência, erro humano ou necessidade de correção. A funcionalidade garantiu maior segurança para as operações críticas e reduziu o risco de impactos negativos no banco de dados.

3.3.4 Timeline

A timeline foi implementada para registrar a sequência de eventos e alterações relacionadas às solicitações de mudança de categoria. Essa visualização ajuda a compreender o

histórico completo do processo, incluindo a origem da solicitação, decisões tomadas e mudanças efetivadas.

3.3.5 Rastreabilidade

A rastreabilidade foi reforçada com o registro dos dados relevantes de cada operação, incluindo informações sobre autor, data, estado atual e histórico de versões. Dessa forma, o sistema passa a permitir auditoria detalhada e maior confiabilidade no acompanhamento das alterações.

3.4 Testes e Validação

3.4.1 Pytest

A validação do módulo foi realizada com a utilização de Pytest, uma ferramenta que permitiu estruturar testes automatizados para verificar o comportamento das funcionalidades implementadas. Esse processo foi essencial para confirmar que as regras de negócio e os fluxos principais continuavam funcionando corretamente.

3.4.2 Pré-configuração

Antes da execução dos testes, foi necessária a configuração do ambiente, incluindo dependências, dados iniciais e parametrização básica do projeto. Essa etapa garantiu que os testes fossem executados em um cenário consistente com o ambiente real de desenvolvimento.

3.4.3 Testes unitários

Os testes unitários foram empregados para validar as partes mais isoladas da lógica, como validações de regras, criação de pedidos e atualização de registros. Essa abordagem aumentou a confiabilidade da implementação e reduziu a chance de erros em componentes específicos.

3.4.4 Testes de integração

Os testes de integração verificaram a comunicação entre os módulos e a correta interação entre backend, banco de dados e interfaces do sistema. Essa camada de validação foi importante para confirmar que as funcionalidades operavam de forma coerente no contexto completo da aplicação.

3.4.5 Processo de Integração Contínua

O processo de integração contínua foi adotado como mecanismo de validação recorrente, permitindo automatizar a execução dos testes e reduzir erros durante o desenvolvimento.

Esse fluxo contribui para manter a qualidade do código e garantir que novas alterações não comprometam o funcionamento do sistema.

4 CONSIDERAÇÕES FINAIS

Cada capítulo deve conter uma pequena introdução (tipicamente, um ou dois parágrafos) que deve deixar claro o objetivo e o que será discutido no capítulo, bem como a organização do capítulo.

Referências

BAYER, M. Sqlalchemy. In: BROWN, A.; WILSON, G. (Ed.). **The Architecture of Open Source Applications, Volume II: Structure, Scale, and a Few More Fearless Hacks**. Lulu.com, 2012. Disponível em: <https://aosabook.org/en/v2/sqlalchemy.html>. Acesso em: 25 out. 2026. Citado na página 5.

DOCKER. **Docker Documentation**. [S.l.], 2024. Disponível em: <https://docs.docker.com/>. Acesso em: 26 out. 2026. Citado na página 7.

FIELDING, R. T.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. [S.l.], 2022. Disponível em: <https://www.rfc-editor.org/info/rfc9110>. Acesso em: 25 out. 2026. Citado na página 4.

FLASGGER DEVELOPERS. **Flasgger: Easy OpenAPI Specs and Swagger UI for Your Flask API**. 2024. Disponível em: <https://github.com/flasgger/flasgger>. Acesso em: 27 out. 2026. Citado na página 5.

FOWLER, M. **Audit Log**. 2004. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/AuditLog.html>. Acesso em: 25 out. 2026. Citado na página 6.

FOWLER, M. **Event Sourcing**. 2005. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/EventSourcing.html>. Acesso em: 25 out. 2026. Citado na página 6.

GAMMA, E. et al. **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading, Massachusetts: Addison-Wesley, 1994. Citado na página 6.

Git. **Git: fast-version-control**. 2026. Disponível em: <https://git-scm.com/>. Acesso em: 27 out. 2026. Citado na página 7.

GitLab. **GitLab Documentation**. 2026. <https://docs.gitlab.com/>. Acesso em: 25 out. 2026. Citado na página 7.

HOLGER KREKEL AND PYTEST DEVELOPERS. **pytest: helps you write better programs**. [S.l.], 2024. Disponível em: <https://docs.pytest.org/>. Acesso em: 25 out. 2026. Citado na página 7.

KLEPPMANN, M. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol, CA: O'Reilly Media, 2017. Citado na página 6.

MARSHMALLOW DEVELOPERS. **marshmallow: simplified object serialization**. [S.l.], 2024. Disponível em: <https://marshmallow.readthedocs.io/>. Acesso em: 25 out. 2026. Citado na página 5.

OpenProject. **OpenProject Documentation**. 2026. <https://www.openproject.org/docs/>. Acesso em: 25 out. 2026. Citado na página 3.

PALLETS. **Flask Documentation**. [S.l.], 2024. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 25 out. 2026. Citado na página 4.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation**. [S.l.], 2024. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 25 out. 2026. Citado na página 5.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Citado 2 vezes nas páginas 6 e 7.

TANENBAUM, A. S.; van Steen, M. **Sistemas Distribuídos: Princípios e Paradigmas**. 2. ed. São Paulo: Pearson Prentice Hall, 2007. Citado na página 4.

Apêndices

APÊNDICE A – Nome do apêndice

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros apêndices para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do apêndice.

Não é aconselhável colocar tudo que é complementar em um único apêndice. Organize os apêndices de modo que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho.

APÊNDICE B – Nome do outro apêndice

conteúdo do novo apêndice

Anexos

ANEXO A – Nome do anexo

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros anexos para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do anexo.

Organize seus anexos de modo a que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho. É para ele que você escreve.

ANEXO B – Nome do outro anexo

conteúdo do outro anexo